Before start, please make sure you've read the following materials:

Retries in Go

How to Make Idempotent APIs in Go: Idempotency-Key, Redis/DB Dedupe, and Race Conditioning

---

Part 1: Retries in Go.

In the world of distributed systems and microservices, network failures, temporary service unavailability, or load spikes are not the exception, but rather the rule. The failure of a single component shouldn't cascade down the entire system. This is where the retry mechanism comes in.

A properly implemented retry allows a system to self-heal from temporary issues, making it more reliable and fault-tolerant. However, improper use of retry can make matters worse by masking serious problems.

> We will consider key patterns, antipatterns and working solutions.

The most naive approach: Infinite loop

Let's start simple. We have a function that can return an error. How can we retry it? The first thing that comes to mind is a simple for loop.

```go
package main

import (
	"errors"
	"fmt"
	"math/rand"
	"time"
)
```

```go
// Simulate an operation that sometimes crashes
func doSomethingUnreliable() error {
	if rand.Intn(10) < 7 {
		fmt.Println("Operation failed, retrying...")
		return errors.New("temporary failure")
	}
	fmt.Println("Operation succeeded!")
	return nil
}

func main() {
	var err error
	for {
		err = doSomethingUnreliable()
		if err == nil {
			break // Success, breaking the loop
		}
		// Error, just try again immediately
	}

	if err != nil {
		fmt.Printf("Failed after multiple retries: %v\n", err)
	}
}
```

> **What's wrong here? Almost everything.**

First, there's no delay between retries. If an error occurs immediately (for example, the network connection is immediately rejected), the loop will run at full speed, eating up CPU and potentially make the problem worse.

Second, there's no limit on the number of retries. If the error isn't transient, this loop will run forever. This is a classic anti-pattern.

Adding a Pause: Fixed Delay

Okay, let's add a pause using time.Sleep.

```go
func main() {
	var err error
	const maxRetries = 5
	const delay = 1 * time.Second

	for attempt := 0; attempt < maxRetries; attempt++ {
		err = doSomethingUnreliable()
		if err == nil {
			break // Success
```

```
        }

        fmt.Printf("Attempt %d failed, waiting %v before next
retry...\n", attempt+1, delay)
        if attempt < maxRetries-1 { // Do not sleep untill the last try
            time.Sleep(delay)
        }
    }

    if err != nil {
        fmt.Printf("Failed after %d attempts: %v\n", maxRetries, err)
    } else {
        fmt.Println("Succeeded within retry limit.")
    }
}
```

We limited the number of attempts and added a fixed delay. But there are nuances here too. A fixed delay may not be optimal. If the problem is resolved quickly, we still wait the full interval. If the problem requires more time, a short delay may not be sufficient, and we continue to overload the system. What if multiple instances of our application start retrying requests simultaneously with the same delay? They will hit the problematic service synchronously, creating a "Thundering Herd" effect.

Exponential Backoff: Growing Exponentially

To solve the problems of fixed latency and synchronization, the `Exponential Backoff` strategy is used. The idea is simple: with each failed attempt, we increase the waiting time, usually by a factor of two.

```
// ... (doSomethingUnreliable remains the same) ...

func main() {
    var err error
    const maxRetries = 5
    baseDelay := 100 * time.Millisecond
    maxDelay := 5 * time.Second

    for attempt := 0; attempt < maxRetries; attempt++ {
        err = doSomethingUnreliable()
        if err == nil {
            break // Success
```

```
        }

        if attempt == maxRetries-1 {
            break // We do not count the delay after the last attempt
        }

        // Calculating exponential delay
        backoffTime := baseDelay * time.Duration(math.Pow(2,
float64(attempt)))
        // Limiting the maximum delay
        if backoffTime > maxDelay {
            backoffTime = maxDelay
        }

        fmt.Printf("Attempt %d failed, waiting %v before next
retry...\n", attempt+1, backoffTime)
        time.Sleep(backoffTime)
    }
    // further logic
}
```

Things have improved significantly. The latency adapts. We start small, but if problems persist, we give the system increasingly more time to recover. The maxDelay limit prevents the latency from growing to absurd levels. However, synchronization issues can still occur, although less frequently.

Jitter: Adding Randomness

When you implement an `exponential backoff`, everything seems fine, but if multiple clients or goroutines start backoffs at the same time, they will do so synchronously—after the same pause.

This leads to the `thundering herd effect`—tens or hundreds of requests arrive simultaneously, overwhelming the service, especially if it's already struggling. To avoid this, randomness is added to the latency—this is called `jitter`. Types of jitter:

Full jitter—complete randomness: the latency is chosen randomly between 0 and the current backoff. It disperses the load as much as possible.

Equal jitter—moderate randomness: half the latency is fixed, the other half is random. It's a balance between stability and randomness.

Decorrelated jitter—an adaptive approach: the next latency depends on the previous one, plus randomness. It works well with a large number of clients, breaking up synchronous peaks.

Let's implement full jitter:

```
// ... (doSomethingUnreliable) ...

func main() {
    var err error
    const maxRetries = 5
    baseDelay := 100 * time.Millisecond
    maxDelay := 5 * time.Second

    rand.Seed(time.Now().UnixNano()) // Improtant for randomness

    for attempt := 0; attempt < maxRetries; attempt++ {
        err = doSomethingUnreliable()
        if err == nil {
            break // Success
        }

        if attempt == maxRetries-1 {
            break
        }

        backoffTime := baseDelay * time.Duration(math.Pow(2,
float64(attempt)))
        if backoffTime > maxDelay {
            backoffTime = maxDelay
        }

        // Adding Full Jitter
        jitter := time.Duration(rand.Int63n(int64(backoffTime)))

        fmt.Printf("Attempt %d failed, waiting ~%v (backoff %v + jitter)
before next retry...\n", attempt+1, jitter, backoffTime)
        time.Sleep(jitter)
    }
    // ... (processing the final result) ...
}
```

Now our retries will be distributed much better over time, reducing the risk of overloading dependent services. `Exponential Backoff + Jitter` is the de-facto standard for reliable retry mechanisms.

Context and Cancellation: Why MaxRetries Isn't Enough

At first glance, setting a `maxRetries` limit might seem to completely solve the problem of infinite retries. However, in reality, this isn't enough. Let's say the user has closed the browser tab, or the external request that initiated the retrieval process has already been canceled.

Despite this, without proper monitoring, we continue to perform retries, wasting system resources by sending unnecessary requests to the database, services, or APIs. These retries are not only useless but also harmful, creating a strain on already overloaded system components.

In production, this is especially critical. That's why Go recommends using `context.Context` as a mandatory part of any retrieval operation.

Context allows for centralized cancellation management, setting deadlines or timeouts, and synchronizing the behavior of different parts of the system.

Using `context.WithTimeout`, you can set a general time limit for an operation, and use `ctx.Done()` or `ctx.Err()` to exit retrieval early if the operation is no longer relevant. This significantly improves the system's resilience and predictability, especially when integrating with other services and asynchronous logic.

```go
type RetryConfig struct {
    maxRetries int
    baseDelay  time.Duration
    maxDelay   time.Duration
}

func Retry(ctx context.Context, cfg RetryConfig) error {
    var err error

    for attempt := 0; attempt < cfg.maxRetries; attempt++ {
        // check for cancellation before each attempt
        if ctx.Err() != nil {
            return ctx.Err()
        }

        err = doSomethingUnreliable()
```

```
        if err == nil {
            return nil
        }

        // if it's the last attempt, we exit with an error.
        if attempt == cfg.maxRetries-1 {
            return err
        }

        // exponential backoff c Full Jitter
        backoff := cfg.baseDelay * time.Duration(math.Pow(2,
float64(attempt)))
        if backoff > cfg.maxDelay {
            backoff = cfg.maxDelay
        }
        jitter := time.Duration(rand.Int63n(int64(backoff)))

        fmt.Printf("Attempt %d failed, waiting ~%v (max backoff:
%v)...\n", attempt+1, jitter, backoff)
        time.Sleep(jitter)
    }
    return err
}
```

Error Filtering: Not Everything Needs to Be Retried

Not every error warrants a retry. It's crucial to distinguish between temporary and permanent errors. Temporary errors include, for example, network timeouts, overloads, rate-limit restrictions, or blocking that can be bypassed by simply waiting.

Permanent errors, on the other hand, include resource not found (404), access denied (403), or the user entering an invalid query. These errors won't disappear on the next attempt, and retrying them wastes resources and potentially worsens the situation.

That's why it's important to evaluate the error itself before each attempt. It's important to clearly understand whether it's retriable. For this purpose, it's convenient to define a helper function, for example, isRetryable(err error) bool, which will encapsulate all the filtering logic.

For example, if you're working with HTTP, you can analyze response codes internally: we only retrieve 429, 500, 502, 503, and 504. If you're working with a database, we check for driver error codes, locks, deadlocks, and connection errors. We ignore syntax errors, duplicate keys, and other signs of business errors.

Go provides convenient tools for such filtering: errors.Is helps compare an error with a known variable, even if it's wrapped multiple times. errors.As extracts the desired error type, such as net.OpError or url.Error, and inspects its fields. This is useful when working with wrapped or compound errors.

What to Avoid: Retry Anti-Patterns

Let's summarize the bad practices:

Retry without delay: Causes excessive overhead.

Retry without limit: Can lead to infinite loops and resource stalls.

Using a fixed delay: Suboptimal and can cause Thundering Herd.

Exponential Backoff without Jitter: Still carries the risk of synchronous retries in high concurrency situations.

Retrying non-temporal errors: Pointless and hides the real problems.

Retrying non-idempotent operations: If retrying an operation would result in duplicate data or unwanted side effects (e.g., resending an email, double-charging), retrying it without additional idempotency mechanisms is dangerous.

Ignoring `context.Context`: Retrying may continue even after the parent operation has been canceled.

Conclusion

The retry mechanism is a powerful tool for building fault-tolerant applications in Go. By understanding the difference between transient and permanent errors, evolving from naive loops to Exponential Backoff with Jitter, integrating with context.Context, and error filtering, we arrive at robust solutions.

In most cases, using proven libraries like `cencalti/backoff` is the preferred path, allowing you to

focus on business logic rather than the intricacies of implementing retry strategies.

Remember: a proper retry isn't simply repeating an action, but a well-thought-out mechanism that improves the reliability of your system. Happy coding!

Part 2: Idempotency.

Imagine a client sending an important POST request (for example, creating an order or debiting funds), but due to a network failure, they didn't receive a response. Unaware that the server has already completed the operation, the client tries to retry the request.

If the backend isn't prepared for such duplicate requests, the outcome could be disastrous: we'll create two identical records instead of one, or worse, charge the user twice. How can this be avoided? The correct answer is to implement idempotency in the API.

Idempotency of a request means that when the same action is invoked multiple times, the system state will only change once.

Simply put, a repeated identical request should not add anything beyond what the first one did. In unreliable network conditions, this mechanism is essential for system resilience. Incidentally, some HTTP methods are considered idempotent by the standard: for example, GET, PUT, and DELETE.

Calling them again should produce the same effect as a single call, provided the server is implemented correctly (not changing state more than once). POST is inherently non-idempotent; two identical POST requests can create two resources. However, we can make POST idempotent by adding a special identifier to each request.

Idempotency Key

To ensure idempotency, the concept of an Idempotency Key, a unique request key, is introduced. This is a unique value generated on the client side and sent to the server along with the request (usually in the Idempotency-Key HTTP header). It is recommended to use a UUID v4 or other random identifier with sufficient entropy to virtually eliminate key collisions. The client should generate a new key for each operation and reuse the same key when retrieving that specific operation (for example, if the same payment needs to be sent again).

Why is this necessary? When receiving a request with such a key, the server will be able to recognize retries and avoid executing the operation twice. The algorithm is generally simple: if a request with a given key has already been executed before, the server returns the previously saved result instead of re-executing the business logic. If the key is seen for the first time, the server executes the request as usual and stores the result, binding it to this key for future use.

For example, what could happen without idempotency: a client sent two requests to debit $1,000 on a loan, and the system executed the operation twice—the final balance decreased to $8,000 (there were two debits of $1,000 each). The duplicate request was processed because the server didn't recognize it as a duplicate. If an idempotency key had been used, this wouldn't have happened—with the second request, the server would have recognized that such a key had already been processed and would have returned the same response to the client without debiting the money again (the balance would have remained $9,000).

Now that the idea is clear, the question arises: how to implement such a mechanism?

The most convenient approach is to move the idempotency logic into a separate layer, such as middleware. That is, write an intermediate handler that intercepts all incoming requests before

the business logic. This idempotency middleware is where we'll check and store our Idempotency Key. This approach allows us to decouple idempotency support from the application's core logic: controllers may not be aware of any keys; the infrastructure layer handles them.

As a result, we achieve a uniform standard for handling repeated requests across the entire API—and a reduced chance of forgetting to enable something.

Idempotency is impossible without stateful server state. We need to store information about previously processed keys and query results somewhere to be able to return a response again. Essentially, a kind of query cache is required: a data structure where a query key can be used to determine whether it was executed and what it returned.

The simplest option is to store this data directly in the application's memory (for example, in a global map). This might work during the initial stages of development: the keys will live as long as the process is running. However, in practice, multiple service instances are usually needed (meaning they need shared memory), and restarting the process will clear the entire map. A repeated request will no longer be recognized after a server restart. Therefore, simple in-memory solutions are quickly moving toward external storage. Let's consider two main options: a relational database and Redis.

---

1. Relational Database

The most reliable way to store data about idempotent queries is in a database. You can create a special table that stores the necessary information for each idempotency_key: execution status and result. For example, the schema might look like this (pseudocode):

```sql
CREATE TABLE idempotency_keys (
    key UUID PRIMARY KEY,
    status VARCHAR(20) NOT NULL,    -- 'processing' or 'completed'
    response_code INT,
    response_body JSONB,
    payload_hash VARCHAR(64) NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    ...
);
```

When a new request with a key arrives, we attempt to insert a new record into this table. By using the unique constraint, the database guarantees that the table won't contain two rows with the same key. However, a simple `INSERT` isn't enough; race conditions must be taken into account.

What if two identical requests arrive simultaneously (for example, on different server instances)? Both will query the database almost simultaneously: they'll see that there's no record with that key, and both will attempt to insert a new row. As a result, one of them may receive a uniqueness violation error (Primary Key conflict), but by then, both may have started executing business logic! We'll end up with the duplicate processing we were trying to protect against.

To prevent this, check/insert operations must be atomic. One approach is to use a transaction with key locking.

For example, in PostgreSQL, you can execute a `SELECT ... FOR UPDATE` on this key within a transaction. If a row with this value doesn't yet exist, the DBMS will place a gap lock on the potential insertion point, forcing a second transaction with the same key to wait. The first query will then insert a row with the `"processing"` status and commit the transaction. The second query, upon waking, will find that the record has already been created. There are several options: you can immediately read the saved result and return it; or, if the result isn't ready yet (we only inserted the status, while the query itself is still in progress), you can return a conflict error or wait. The key is that both parallel attempts don't execute business logic.

Ultimately, only one of them will actually create a resource or perform an action, while the second simply learns the result of the first. After the first query completes, we update the record in the database, setting the status to `"completed"` and storing the result (for example, code 201 and the response body). Repeated requests with the same key will be able to immediately retrieve these saved `response_code` and `response_body` from the database, without having to re-execute the operation.

To summarize, the database guarantees unique keys and, through transactions, allows for the proper ordering of concurrent requests. The downside is the overhead: each idempotent request will require at least one database roundtrip (or even two or three if using `SELECT` + `INSERT` + `UPDATE`). If the load is high, it might be worth considering a lighter storage solution.

---

2. Redis (or another fast cache)

A very popular solution is to use Redis to store idempotency state.

We can store `key -> result` pairs in it. Furthermore, Redis is accessible to all instances of our service simultaneously, which is important for a distributed application. The downside is that in-memory data is less durable: if Redis is restarted or the cache is reset, information about processed requests will disappear.

However, complete durability isn't always necessary for idempotency; storing keys for a relatively short period of time (say, 24 hours) is usually sufficient. The likelihood that a client will accidentally repeat the same request days or weeks later is negligible. Therefore, many choose Redis as a compromise: data is short-lived but very fast to access.

How to avoid race conditions in Redis? Don't try to perform separate `GET` and then `SET` operations; another request could intervene between them. The solution is to use Redis's atomic operations. In our case, the `SET` command with the NX (Not eXists) option is ideal; it only writes to keys that don't already exist, in a single operation.

We can do, for example:

```
SET idempotency:<key> "processing" EX 60 NX
```

This single request to Redis will attempt to set the `"processing"` value for a new key with a TTL (time to live) of 60 seconds only if such a key doesn't already exist. Using the `NX` flag, Redis ensures that only one concurrent request will be able to create this key.

The first request to enter the system will successfully write its key and effectively acquire the lock for the duration of its execution. The second concurrent request with the same key will receive a response from Redis that the key already exists and will know it has been beaten to the punch.

The scenario then resembles the database case, although implemented slightly differently. The first request wrote the value "processing" to Redis (indicating that work had begun) and proceeded to execute the business logic. The second request discovered that the key existed. What should it do? It could try waiting a few seconds and then GET this key. If the value is still "processing," then the first request hasn't finished yet—it can return a 409 Conflict error to the client (indicating that the request is already in progress).

Some implementations do this right away: the duplicate request immediately receives a 409 error. An alternative approach is for the server to wait a while for the first request to complete. For example, you could check Redis again after a few moments: if the first request has already written the final result (more on this below), immediately return it to the second client. If, however,

there's still no result after the timeout, return a 409 Conflict or a 429 Too Many Requests error with a Retry-After header. This hybrid approach makes repeat requests more transparent to the client: often, the second request will simply wait an extra 100–200 ms and receive a response as if it had performed the operation itself.

We're getting a bit ahead of ourselves. Let's return to the first request: after executing the business logic, it must save the result to storage. In the case of Redis, this means storing the serialized response (status + body) in the key, not "processing."

You can either update the same key or set a new key of the form `idempotency:<key>:result`. For simplicity, the value is often overwritten: what was `"processing"` becomes, for example, `"{\"status\":201,\"body\":\"...\"}"` with a longer TTL (e.g., 24 hours). Now Redis stores the final response.

When a repeat request with the same key arrives (even a second later, even to a different server), we perform a `GET` and retrieve the resulting value. The server will return this cached response instead of re-executing the business logic. This ensures idempotency.

We set a TTL of 60 seconds on the `"processing"` label. This is a kind of safety timer: if the first request fails or hangs, the key will be deleted automatically after a minute, preventing the system from freezing permanently. Of course, the timeout here should be chosen with a margin (the operation execution time plus a little extra). For the final result, it makes sense to set a longer TTL—hours or days. Usually, 24 hours is sufficient: keys older than 24 hours can be considered `"outdated"` and deleted to prevent them from accumulating indefinitely (Stripe does exactly this: keys are stored for at least 24 hours, after which they can be cleared).

Atomicity and Race Conditions

As we've already seen, the key to the implementation is to properly handle race conditions when two requests with the

same key arrive almost simultaneously. A naive implementation won't work here. If storage checks and updates are separated, concurrent requests can easily bypass our logic and both decide they're "first." The solution is atomic operations. In the case of a database, these are transactions/locks; in the case of Redis, these are commands like `SETNX` or Lua scripts that perform the check and write simultaneously. Always try to use either a single atomic command or a lock during the key check—then only one thread will "win" and execute the operation. Otherwise, the purpose of idempotency is lost.

To be fair, a global lock on the key (for example, an application-level mutex) would also solve the race condition, but it can become a bottleneck. It's better to rely on the atomicity of the storage itself, as described above, especially if the API scales across multiple instances.

Middleware for idempotency in Go

We'll write a simplified version of the middleware in Go that implements the described ideas. We'll use standard `net/http`. For simplicity, we'll implement an in-memory key store (on a single server); if desired, it can easily be replaced with a Redis or database implementation. Our middleware will check the `Idempotency-Key` header for each resource creation request and handle duplicates.

```go
package main

import (
    "net/http"
    "net/http/httptest"
    "sync"
)

// Structure for storing cached response
type CachedResponse struct {
    StatusCode int
    Body       []byte
    Completed  bool
}

type MemoryStore struct {
```

```go
    mu   sync.Mutex
    data map[string]*CachedResponse
}

func NewMemoryStore() *MemoryStore {
    return &MemoryStore{ data: make(map[string]*CachedResponse) }
}

// Get stored response by ket (if exists)
func (m *MemoryStore) Get(key string) (*CachedResponse, bool) {
    m.mu.Lock()
    defer m.mu.Unlock()
    resp, exists := m.data[key]
    return resp, exists
}

// Try to reserve ket for new request
func (m *MemoryStore) StartProcessing(key string) bool {
    m.mu.Lock()
    defer m.mu.Unlock()
    if _, exists := m.data[key]; exists {
        return false  // key already exists
    }
     // Insert an "empty" record, marking that the request is in progress
    m.data[key] = &CachedResponse{ Completed: false }
    return true
}

// Save the result and mark the query as completed
func (m *MemoryStore) Finish(key string, status int, body []byte) {
    m.mu.Lock()
    if resp, exists := m.data[key]; exists {
        resp.StatusCode = status
        resp.Body = body
        resp.Completed = true
    } else {
        // just in case there was no entry (shouldn't happen)
        m.data[key] = &CachedResponse{ StatusCode: status, Body: body,
Completed: true }
    }
    m.mu.Unlock()
}

// Middleware implementing idempotency
func IdempotencyMiddleware(store *MemoryStore, next http.Handler)
http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        key := r.Header.Get("Idempotency-Key")
        if key == "" {
            http.Error(w, "Idempotency-Key header required",
http.StatusBadRequest)
            return
        }

        // Check if this key has already been processed
        if cached, exists := store.Get(key); exists {
            if cached.Completed {
```

```
                // Return the saved result
                w.WriteHeader(cached.StatusCode)
                w.Write(cached.Body)
            } else {
                // A request with this key is still being processed
                http.Error(w, "Duplicate request in progress",
http.StatusConflict)
            }
            return
        }

        // Trying to reserve the key (mark as "in progress")
        if !store.StartProcessing(key) {
            // Someone managed to insert the key before us.
            if cached, exists := store.Get(key); exists &&
cached.Completed {
                // If the request has just completed, return the result
                w.WriteHeader(cached.StatusCode)
                w.Write(cached.Body)
            } else {
                // Otherwise, we will report a conflict (duplicate in
progress)
                http.Error(w, "Duplicate request in progress",
http.StatusConflict)
            }
            return
        }

        // Our first request is to execute the main logic
        recorder := httptest.NewRecorder()
        next.ServeHTTP(recorder, r)

        // Save the result and mark the key as complete
        store.Finish(key, recorder.Code, recorder.Body.Bytes())

        // Return to the client the response received from the main logic
        for k, vals := range recorder.Header() {
            for _, v := range vals {
                w.Header().Add(k, v)
            }
        }
        w.WriteHeader(recorder.Code)
        w.Write(recorder.Body.Bytes())
    })
}
```

We created a simple `MemoryStore` with a mutex that stores request information. In reality, as discussed, it would be better to use external storage instead of memory, but the interface would be roughly the same.

After this, the query is complete. If a repeat request with the same key is received, it will be placed in the `exists && Completed`

branch and immediately receive the cached result.

Our example is designed for a single application instance. In a multi-server environment, MemoryStore should be replaced with something like RedisStore or DBStore, but the principles remain the same. For example, `StartProcessing` can be implemented using the `SETNX` command in Redis, as discussed earlier. The pseudocode is as follows:

```
res, err := redis.SetNX(ctx, key, "processing", 5*time.Minute).Result()
if err != nil { /* Redis connection error */ }
if res {
    // the key was successfully installed - we are the first request
} else {
    // key already exists
    val, _ := redis.Get(ctx, key).Result()
    if val == "processing" {
        // another request is still in progress -> 409 Conflict
    } else {
        // val contains the ready answer -> return it
    }
}
```

Here we set `"processing"` with a TTL of 5 minutes, similar to our `StartProcessing`. Then, if `SetNX` returns `false`, we read the current value. If it already contains the final result (for example, JSON), we return it immediately; if it's still "processing," we reject the request. The first request upon completion should execute something like:

```
redis.Set(ctx, key, <serialized_response>, 24*time.Hour)
```

to save the result. In the case of a database, the code will look different (using `INSERT … ON CONFLICT DO …` or a transaction), but the idea is the same: use key uniqueness and transactionality.

Individual Assignment (3 points)

№1: Resilient HTTP Client

Imagine you're developing an Order Service for an online store. When a user clicks "Pay," your service needs to call an external Payment Gateway API.

The problem is in production:

During sales (like Black Friday), the payment gateway starts to overload. It doesn't crash completely, but it returns `429 Too Many Requests` or `503 Service Unavailable` errors. If you don't use Retry with the right timing (Backoff + Jitter), you'll either lose the customer's money or overwhelm the payment gateway with a barrage of repeated requests, preventing it from recovering.

Task

Implement a retry mechanism for a payment processing system that must withstand temporary failures of an external API.

1. Filtering and Calculation Logic

Implement two functions that define the system's behavior:

`IsRetryable(resp *http.Response, err error) bool`: should return `true` if the error is temporary (network timeout, codes 429,

500, 502, 503, 504). If the error code is 401 (invalid API key) or 404, retrying is useless and returns `false`.

`CalculateBackoff(attempt int) time.Duration`: implements exponential backoff with added Full Jitter (randomization) to distribute the server load.

2. Implementing a Fault-Tolerant Client

Write an `ExecutePayment` method that:

1. Accepts a `context.Context` with a global timeout for the entire operation (e.g., 10 seconds).

2. Executes the HTTP request.

3. If the request is unsuccessful and the error is retriable, it waits the required amount of time and tries again.

4. Important: If the context is terminated (timeout or cancel), the loop must terminate immediately, without waiting for the next attempt.

3. Simulation of a “Production Situation”

In the main function, you need to simulate an unstable server using `httptest`.

Interaction Scenario:

1. Create a test server that:

- Returns 503 Service Unavailable for the first 3 requests.
- Returns 200 OK and JSON {"status": "success"} for the 4th request.

2. Configure your client to:

- MaxRetries: 5
- BaseDelay: 500ms

3. Run the execution. The logs should clearly show:

- "Attempt 1 failed: waiting 500ms..."
- "Attempt 2 failed: waiting 1200ms..." (example of jittered latency)

- "Attempt 4: Success!"

№2: Loan Repayment

Imagine a bank's mobile app. The user clicks the "Pay $1000" button. The request is sent to the server, which successfully debits the money, but at that moment, the user's internet connection is lost. The app doesn't receive a response and, following the `Retry` logic (from the first part), resends the request with the same idempotency key.

Your task: implement a layer that recognizes this repeated request and, instead of debiting it again, simply returns the result of the previous (successful) operation.

---

1. Idempotency Middleware Implementation

Implement middleware that intercepts requests and checks the `Idempotency-Key` header.

- Storage: Use Redis or Database
- Logic:

1. If the `Idempotency-Key` header is missing, return `400 Bad Request`.

2. If a request with this key is already in progress (`Status: processing`), return `409 Conflict`.

3. If a request with this key has already completed (`Status: completed`), return the saved status code and response body without calling the main business logic.

4. If the key is new, start execution, save the result, and return it to the client.

2. Simulating long-running business logic

In the main payment handler, implement a simulation of real work:

- Perform a "heavy" operation using `time.Sleep(2 * time.Second)`.
- Return JSON of the form `{"status": "paid", "amount": 1000, "transaction_id": "uuid-..."}`.
- This is necessary so that while the first request is being processed, we have time to send the second one and test the `409 Conflict` handling.

3. Simulating a "Double-Click" Attack

In the main function (or a separate script), implement a scenario simulating a production situation:

1. Run 5-10 requests to your server simultaneously (using goroutines) with the same `Idempotency-Key` value.

2. The logs should clearly show:

- One request has started executing ("Processing started").
- The remaining requests received within these 2 seconds received a `409 Conflict` response.
- After the first request completed, the next request with the same key received a `200 OK` response with the same JSON that was generated the first time (the business logic was not re-executed).

Demo video scenario

- Student runs all solutions separately for each task(1 and 2) and examples from parts 1 and 2.

- Student demonstrates the result of execution of each task from terminal

Definition of Done (DOD)

- All tasks are accomplished according to the instructions, and all the examples from parts 1, 2 are implemented by

student
- All solutions executes with a successful status
- `Demo video` is recorded according to the `Demo video scenario`

Deliverables:

- Push all the changes to your GitHub repository
- Submit the link to this repository(Please, submit the link as a LINK ATTACHMENT, not a file)
- Submit [1-2]-minute demo video of your project as a VIDEO-FILE ATTACHMENT. You can speed it up if it is too long.

> ## Below are marking criteria:

Criteria	Weight
Problem 1	1.5 points
Problem 2	1.5 points
OVERALL	3 points

> ## Below are penalties criteria:

Adherence to all the requirements above is mandatory. Failure to implement any of the specified points (e.g., missing `sync.WaitGroup` or neglecting `context` support) will result in a significant point deduction.

Category	Criteria	Penalty
Video Demo	Video Demo is does not Present or Unable to Open/View or Contradicts ethical standards or Contradicts the Demo video scenario	3 points
Video Demo Link	The video demo is specified in any way other than the `video-file` attachment	Personal Defence
Git Repo Link	The link to the git repository is specified in any way other than the `link-attachment`	Personal Defence
Git Repo Content	The content of git repository for this practice is corrupted or unable to Open/Read	3 points

Category	Criteria	Penalty
Your Task part	At least one requirement (e.g. Demonstrate the execution of the code using -race flag.) from `Individual Assignment` part is failed to meet	100% of a problem's solution mark
Cheating/ Malpractice	Students practicing cheating, duplicating the contents of other students solutions	100% of the `Overall` mark for each of the students
Definition of Done	At lease one criteria from `Definition of Done` is failed to meet	100% of the `Overall` mark

---

> ## DEADLINE: 26.04.2026, SUNDAY 23:59

OPTIONAL FEATURES

> **Students may implement any number of the following to increase difficulty and engagement. Your optional features will be considered only if you demonstrated them in your demo video and explained it in details.**

Implement any features of your choice related to the topics discussed during this week.